Stellar Cyber 6.6.0s is now live! Discover all the updates in the release notes!

Deploying Linux Server Sensors Using Ansible

6.4.0

This topic describes how to deploy Linux Server Sensors using an Ansible playbook and inventory file downloaded from Stellar Cyber. Once customized, you can run the playbook from your own Ansible administration system..

See the following sections for details:

- **Downloading Sample Ansible Files**
- **Customizing the Ansible Playbook for Linux Server Sensor Deployment**
- **Sample Ansible Files**

Downloading Sample Ansible Files

You can download a sample Ansible playbook and inventory file using the links below. You can see the contents of the sample playbook and inventory file in **Sample Ansible Files**.

Contact Stellar Cyber support (support@stellarcyber.ai) for login credentials.

Sample Playbook File	`curl -O -k -u login:password https://acps.stellarcyber.ai/public/aella-agent-install.yml`
Sample Inventory File	`curl -O -k -u login:password https://acps.stellarcyber.ai/public/aella-agent-inventory`

Customizing the Ansible Playbook for Linux Server Sensor Deployment

Before running the playbook, use the information in the table below to review and update the variables in the `vars` section to match your deployment environment. These values control how the installer is downloaded, authenticated, and registered with the Stellar Cyber platform.

Variable	Required	Description	Example
`cloud_url`	Yes	Hostname used to download the Linux Server Sensor installer. Do not change this variable. It is preconfigured with the address of the Stellar Cyber software repository (**acps.stellarcyber.ai**).	`acps.stellarcyber.ai`
`version`	Yes	Sensor software version to install. This value is used to construct the installer download URL. The playbook downloads the installer from: **https://<cloud_url>/release/<version>/datasensor/**	`6.6.0`
`token`	Conditional	**SaaS Deployments** – Registration token used to onboard the sensor. If this value is provided, the installer uses token-based authentication and adds itself to the Data Processor encoded in the token. You obtain tokens in the **System | DATA SOURCE MANAGEMENT | Sensors | Sensor**	Click to see: "eyJhbGciOiJkaXIiLCJlbmMiOiJBMTI4R0NNIn0..S_4moX-NTXCOy2adE7q5Jw.3hSjeblj9dzqrcGduttYHY6adfcs054-yclhXS94anVs84TUBNY_dMayEy34o50U9vvrIRKOXrMRXJPs9CElKGIWeHlWLYanyiIXR1phe1R-TeXpKtfJUZ9k7ZuWjzr6wLOprM3EdGgxs4LuOCwAGs4WltGCu5DcHg1ZEWXj5BKC4oQwj37J5dGxRw4qpAo3cO1_A2hHkVmS3m21it7v9KMefEt1UVJluFzRlRpBCh6BwwuNuPtQ8KjW5OkmABwj9rjP03iOW4svlx8AEkJ5jkwG3VzCDaeIL_Ao_fRMdL9tR64AOzuN4f5x_YMVkV_7PjRE6rKSgMQmq83-2xVvENjdjzyFu2cUSddnOGPgEoK5EaZWo1mlXGhOVwqVkojTyDqb0F7QUvD_mU4Wg5_Si-Rrh3b8fAB-iwbbPUKxQt0AH8-HSbDyNJYfecB_1mAHLYaWYzxdtOW7rxVRIyqEYXy6517NlKwaEUw5MSWBZfWMOg0FKCnH-J-YrMgpC1jcDNjukqbs7wUAdmVoXhjQsAILfQHeD_rIlnO0bEEOsYb9fbxVKNAuZyMMpZYgomBJu6LKNzKcvvSoPoXShy_2hG0-zWQEcpxgcEAPXcncjNOnGnpgVR76uniEis8HfF78CjcFqQl_e2hg9vash8ohYFihif-8FPbpFA9lRuhZU8lFk6aKz-Zo1YOunXJVtA.QJ8dSftCckeD8i3tMu9UXw"

Variable	Required	Description	Example
		Installation | Tokens. Refer to **Obtaining a Token for the Installation** for details.	
`cm_ip`	Conditional	**On-Premises Deployments** – Stellar Cyber If a token is not provided, the installer uses the Data Processor IP address specified here for sensor registration.	`192.168.11.11`
`ds_username`	Yes	Username used to authenticate when downloading the installer from **acps.stellarcyber.ai**. Contact Stellar Cyber support (support@stellarcyber.ai) for login credentials.	`"download_user"`
`ds_password`	Yes	Password used to authenticate when downloading the installer from **acps.stellarcyber.ai**. Contact Stellar Cyber support (support@stellarcyber.ai) for login credentials.	`"download_password"`
`self_contain`	Yes	Determines which installer script is used: • `true` – The all-in-one installer **(ds_linux_inst**	`true`

Variable	Required	Description	Example
		all_all_in_one.sh) is used. • `false` – The run-time installer (**ds_linux_install.sh**) is used. Refer to **Choosing an Installation Script with the self_contain Variable** for details on the differences between the two scripts. If you are not sure which installer to select, use the all-in-one script (the default).	
`install_dir`	No	Directory on the target host where the installer will be downloaded. Set to `/tmp` by default.	`/tmp`
`tenant_id`	No	Associates the sensor with a specific tenant.	`"My-Tenant"`
`primary_aggregator`	No	Primary aggregator used by the sensor for communication with the Stellar Cyber platform.	`"10.0.1.10"`
`secondary_aggregator`	No	Backup aggregator used if the primary aggregator becomes unavailable.	`"10.0.1.11"`

Choosing an Installation Script with the self_contain Variable

Stellar Cyber provides two different installation scripts for the Linux Server Sensor:

- **ds_linux_install_all_in_one.sh** is an entirely self-contained installation script that includes the images for all supported target environments. Because of this, it does not require access to the Internet and can be used in dark sites. However, it is quite large (on the order of ~700 MB).
- **ds_linux_install.sh** does not bundle images with the script and instead pulls the correct image from the Stellar Cyber production server at run time. Because of this, it is much smaller than the all-in-one script (on the order of 10 KB, plus the single 200 MB image pulled from Stellar Cyber production servers).

The table below provides you with details on the two scripts:

Name	Type/Size	Usage	Dark Site Support	Pros	Cons
ds_linux_install.sh	Small Installer does not include images and is roughly 10 KB. If used without the `-p/--package` parameter, the single image pulled from Stellar Cyber production servers is roughly 200 MB.		Yes, but only if the installation image is downloaded separately, copied to the target machine, and pointed to with the `-p/--package` parameter at run time.	Script is small and only downloads the one image required for installation.	Internet access required in most use cases.
ds_linux_install_all_in_one.sh	Large Installer includes images for all supported target environments and is roughly ~700 MB.	Mostly identical. Obtain credentials from Customer Support, download and execute script with optional arguments.	Yes, with no further download.	Supports dark site installation with no further downloads.	Script is large.

Customizing the Ansible Inventory File

The Ansible inventory file defines the target systems where the Linux Server Sensor will be installed. The playbook references the `target` host group, so any hosts included in that group will receive the deployment.

The inventory file contains the sections shown below:

```
[hosts_linux]
10.36.29.162

[hosts_linux:vars]
ansible_connection=ssh
ansible_port=22
ansible_user=root
ansible_ssh_pass=mypassword
ansible_become_pass=mypassword
ansible_ssh_common_args='-o UserKnownHostsFile=/dev/null -o
StrictHostKeyChecking=no'

[target:children]
hosts_linux
```

Host Group Definition (hosts_linux Section)

The `hosts_linux` group defines the Linux servers where the sensor will be installed. Each entry represents a target host.

```
[hosts_linux]
10.36.29.162
```

To deploy the sensor to additional systems, add their hostnames or IP addresses to this list. For example:

```
[hosts_linux]
10.36.29.162
10.36.29.163
10.36.29.164
```

Connection Variables (hosts_linux:vars Section)

The `hosts_linux:vars` section defines connection settings used by Ansible when connecting to the target systems.

Variable	Description	Example
`ansible_connection`	Connection method used to access the host.	`ssh`
`ansible_port`	SSH port used for the connection.	`22`
`ansible_user`	User account used to log in to the target system.	`root`
`ansible_ssh_pass`	Password used for SSH authentication.	`mypassword`
`ansible_become_pass`	Password used for privilege escalation when running commands with sudo.	`mypassword`
`ansible_ssh_common_args`	Optional SSH arguments. In this example, strict host key checking is disabled to simplify initial connections.	`-o UserKnownHostsFile=/dev/null -o StrictHostKeyChecking=no`

Target Group Mapping (target:children Section)

The playbook installs the Linux Server Sensor on the `target` host group. The inventory file maps the `hosts_linux` group into the `target` group using the following configuration:

```
[target:children]
hosts_linux
```

This structure allows you to organize hosts into logical groups while keeping the playbook consistent. Any host placed in `hosts_linux` automatically becomes part of the `target` group used by the playbook.

Running the Deployment

Once the inventory file and playbook variables are configured, you can run the deployment:

- Run the deployment from an Ansible control node as follows:

  ```
  ansible-playbook -i aella-agent-inventory aella-agent-install.yml
  ```

Ansible will connect to each host defined in the `target` group and install the Linux Server Sensor automatically.

For production deployments, consider using SSH key authentication instead of storing passwords directly in the inventory file.

Sample Ansible Files

The following files are provided to illustrate how the Ansible files might appear. Refer to **Customizing the Ansible Playbook for Linux Server Sensor Deployment** for details on how to customize the files for your own deployment.

Sample Ansible Playbook File

The playbook file is a YAML-format file similar to the following:

```
---

- hosts: target
  become_method: sudo
  become: yes
  gather_facts: true
  vars:
     # Optional token - if set, will use --token; otherwise uses --cm
     token: ""
     cm_ip: 192.168.11.11
     cloud_url: acps.stellarcyber.ai
     version: 6.2.0
     self_contain: true
     ds_username: "user310"
     ds_password: "HMTe3dJ3cmAPK"
     install_dir: /tmp
     # Optional parameters
     tenant_id: ""                # e.g., "123456"
     primary_aggregator: ""       # e.g., "10.0.1.10"
     secondary_aggregator: ""     # e.g., "10.0.1.11"
  tasks:
    - name: Choose installer script based on self_contain
      set_fact:
        installer_script: >-
          {{ 'ds_linux_install_all_in_one.sh' if (self_contain | bool) else
'ds_linux_install.sh' }}

    - name: Announce installation
      debug:
        msg: >-
          Installing Data Sensor Agent (Linux) using {{ installer_script }} — please
wait...

    - name: Download installer
      ansible.builtin.get_url:
        url: "https://{{ cloud_url }}/release/{{ version }}/datasensor/{{
installer_script }}"
        dest: "{{ install_dir }}/{{ installer_script }}"
        url_username: "{{ ds_username }}"
        url_password: "{{ ds_password }}"
        force: yes
        validate_certs: no
        mode: "0755"

    # Conditionally choose install flag (token or cm)
    - name: Determine installer authentication flag
      set_fact:
        auth_flag: >-
          {% if token is defined and token | trim | length > 0 -%}
          --token {{ token | quote }}
          {%- else -%}
          --cm {{ cm_ip }}
          {%- endif %}

    - name: Build optional installer flags
      set_fact:
        optional_flags: >-
          {% set flags = [] -%}
          {% if tenant_id is defined and tenant_id | trim | length > 0 %}
```

```
          {%   set _ = flags.append('--tenant_id ' ~ tenant_id | quote) %}
          {% endif %}
          {% if primary_aggregator is defined and primary_aggregator | trim | length
> 0 %}
          {%   set _ = flags.append('--primary_aggregator ' ~ primary_aggregator |
quote) %}
          {% endif %}
          {% if secondary_aggregator is defined and secondary_aggregator | trim |
length > 0 %}
          {%   set _ = flags.append('--secondary_aggregator ' ~ secondary_aggregator
| quote) %}
          {% endif %}
          {{ flags | join(' ') }}
    - name: Debug optional flags (safe for logs)
      debug:
        msg: "Optional flags passed: {{ optional_flags | default('(none)') }}"

    - name: Echo installer command
      debug:
        msg: >
          bash {{ install_dir }}/{{ installer_script }}
          -v {{ version }}
          {{ auth_flag }}
          {{ optional_flags | default('') }}

    - name: Run installer
      ansible.builtin.shell: >
        bash {{ install_dir }}/{{ installer_script }}
        -v {{ version }}
        {{ auth_flag }}
        {{ optional_flags | default('') }}
      args:
        warn: false

    - name: Display post install message
      debug:
        msg:
          - "Stellar Cyber Linux Agent Sensor installation complete."

    - name: Wait a bit for CM to start sending configuration
      ansible.builtin.pause:
        seconds: 15
        prompt: "Pausing 15 seconds to allow CM to register the agent..."

    - name: Wait until cm_assigned_name is assigned (not 'unknown')
      shell: egrep 'cm_assigned_name *= *' /etc/aella/aos.yaml | awk '{print $4}'
      register: device
      changed_when: false
      retries: 15          # roughly 150 secs (15 * 10s)
      delay: 10
      until:
        - device.stdout is defined
        - device.stdout | length > 0
        - device.stdout != 'unknown'

    - name: Display Agent Assigned Name from CM
      debug:
        var: device.stdout

    - name: Get engid
      ansible.builtin.shell: "egrep 'engid *= *' /etc/aella/aos.yaml | awk '{print
$4}'"
      register: engid
      changed_when: false

    - name: Display engid
      debug:
        var: engid.stdout
```

Sample Inventory File

A sample Ansible inventory is shown as follows:

```
[hosts_linux]
10.36.29.162

[hosts_linux:vars]
ansible_connection=ssh
ansible_port=22
ansible_user=root
ansible_ssh_pass=mypassword
ansible_become_pass=mypassword
ansible_ssh_common_args='-o UserKnownHostsFile=/dev/null -o
StrictHostKeyChecking=no'

[target:children]
hosts_linux
```


© 2026 Stellar Cyber . All rights reserved.